

Reviewer Pack — Fourier Coefficient Concentration in ACC^0 Circuits

Entry 971d5ab3c3ef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 04:17:44 UTC

Fourier Coefficient Concentration in ACC^0 Circuits

Entry ID: 971d5ab3c3ef **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 04:17:44 UTC

1. Conjecture

Field A (mathematical branch): FOURIER_ANALYSIS **Field B** (complexity object): ACC_ZERO_CIRCUIT_COMPLEXITY

Statement:

A Boolean function f on n variables cannot be computed by an ACC^0 circuit of size $o(2^{\{n/2\}})$ if its Fourier transform has a coefficient $|F(S)| \geq 1/\sqrt{n}$ for some subset S of size $\Omega(n)$.

Rationale (proposer’s reasoning):

This conjecture connects ACC^0 circuit complexity to the structure of Fourier coefficients, leveraging the known lower bounds for functions with large Fourier coefficients. If true, it would provide a natural property (Fourier concentration) that excludes ACC^0 circuits while being computable in subexponential time via Fourier transform analysis on small n .

Taxonomy category: ACC_SIPSER (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 232859c8201b5771

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = lambda x: sum(x[i] for i in range(n)) % 2 # Example function
    ft = fourier_transform(f, n)

    max_coefficient = max(abs(coeff) for coeff in ft)
    conjecture_holds = max_coefficient < 1 / math.sqrt(n)
    counterexample = "" if conjecture_holds else f"Max coefficient: {max_coefficient}"

    return {
        "metric_name": "max_fourier_coefficient",
        "metric_value": max_coefficient,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def fourier_transform(f, n):
    result = [0] * (1 << n)
    for S in range(1 << n):
        sum_val = 0
        for x in range(1 << n):
            term = f(x) * math.exp(-2j * math.pi * S * x / (1 << n))
            sum_val += term
        result[S] = sum_val / math.sqrt(1 << n)
    return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"max_coefficient\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE support_fraction_too_low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aele000f.py", line 52, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aele000f.py", line 22, in run_trial
    ft = fourier_transform(f, n)
         ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aele000f.py", line 41, in fourier_transform
    term = f(x) * math.exp(-2j * math.pi * S * x / (1 << n))
           ~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aele000f.py", line 21, in <lambda>
    f = lambda x: sum(x[i] for i in range(n)) % 2 # Example function
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aele000f.py", line 21, in <genexpr>
    f = lambda x: sum(x[i] for i in range(n)) % 2 # Example function
                   ~~~^
TypeError: 'int' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in Fourier transform computation, preventing data collection. The crash indicates a code error rather than a theoretical failure. | next: Fix the test code to handle x as a list/array instead of integer, then re-run with proper input types

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	70026
2	propose	ollama_remote	qwen3:8b	0	0	90362
3	propose	ollama_remote	qwen3:8b	0	0	65778
4	propose	ollama_remote	qwen3:8b	0	0	86029
5	preregistration	ollama_remote	qwen3:8b	0	0	27260
6	novelty	ollama_remote	qwen3:8b	0	0	23908
7	novelty	ollama_remote	qwen3:8b	0	0	18282
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12882
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8907
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7026
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6857
12	judge	ollama_remote	qwen3:8b	0	0	21517

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 438833 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in *research/audit/971d5ab3c3ef.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/971d5ab3c3ef.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/971d5ab3c3ef.tar.gz* (if generated)